

Raport Temă Practică ML

Aldea Alexandra-Maria, Cotea Carla-Gabriela

Ianuarie 2026

1 Preprocesare (Task 2.1)

1.1 Clasificarea Produselor

Scop

Primul pas a fost împărțirea produselor pe categorii generalizate din produsele particulare din datasetul original (de exemplu, “Pepsi”, “Crazy Schnitzel”, “French fries”). De asemenea, s-au adăugat coloanele de `tip_zi` (weekday/weekend) și `perioada_zi` (morning/lunch/evening).

Implementare

Scriptul `add_category_column.py` definește o funcție care primește numele produsului și, pe baza unor liste predefinite, atribuie o categorie.

```
def categorize_product(product_name):
    if 'Pepsi' in product_name or 'Aqua' in product_name:
        return 'drink'
    elif product_name in ['Crazy_Sauce', 'Cheddar_Sauce', ...]:
        return 'sauce'
    elif 'Schnitzel' in product_name:
        return 'schnitzel'
    # ... etc
```

Categorii Definite

Am împărțit toate produsele în 9 categorii semantice:

- **drink**: Băuturi (Pepsi, Aqua, Lipton, etc.)
- **sauce**: Sosuri comandate standalone (Crazy Sauce, Cheddar Sauce, etc.)
- **schnitzel**: Schnitzele
- **side_dish**: Garnituri (cartofi)
- **main_dish**: Fel principal (Mac & Cheese, Chicken Bao Buns)
- **extra**: Ingrediente extra
- **salad**: Salate
- **dessert**: Deserturi
- **packaging**: Ambalaj

Exemplu după categorizare

Tabela 1: Exemplu date după categorizare

id_bon	Product Name	Category	tip_zi	perioada_zi
1	Crazy Schnitzel	schnitzel	weekday	lunch
1	Crazy Sauce	sauce	weekday	lunch
1	Pepsi	drink	weekday	lunch
2	Crazy Schnitzel	schnitzel	weekend	evening
2	French fries	side_dish	weekend	evening

1.2 Agregare la Nivel de Bon

Problema

După pasul anterior, datele sunt la nivel de produs individual. Pentru a antrena un model de clasificare la nivel de bon, trebuie agregate informațiile tuturor produselor pe fiecare bon.

Transformarea Datelor

Scriptul `create_modified_dataset.py` transformă datele prin:

1. Grupare după `id_bon`.
2. Pentru fiecare bon, numărare a produselor per categorie.
3. Pentru fiecare bon, numărare a fiecărui produs specific.
4. Creare de coloane noi: `count_category_*` și `count_product_*`.

Exemplu Transformare

ÎNAINTE (nivel de produs):

id_bon	retail_product_name	product_category
1	Crazy Schnitzel	schnitzel
1	Crazy Sauce	sauce
1	Pepsi	drink
2	Crazy Schnitzel	schnitzel
2	French fries	side_dish

DUPĂ (nivel de bon):

id_bon	count_cat_schnitzel	count_cat_sauce	count_prod_sauce	...
1	1	1	1	
2	1	0	0	

1.3 Filtrare și Extragere Features Finale

Context

Scriptul `filter_crazy_schnitzel+features.py` realizează următoarele:

1. Filtrează doar bonuri care conțin *Crazy Schnitzel*.
2. Creează eticheta target.
3. Computează caracteristici de coș.

1. Filtrare Inițială

```
bonuri_cu_crazy_schnitzel = df[df['retail_product_name'] == 'Crazy_Schnitzel']['id_bon'].unique()
df = df[df['id_bon'].isin(bonuri_cu_crazy_schnitzel)]
```

2. Eticheta Target (y)

```
bonuri_cu_crazy_sauce = df[df['retail_product_name'] == 'Crazy_Sauce']['id_bon'].unique()
df['y'] = df['id_bon'].apply(lambda x: 1 if x in bonuri_cu_crazy_sauce else 0)
```

Variabila target este binară:

- $y = 1$: Bonul conține *Crazy Sauce*.
- $y = 0$: Bonul NU conține *Crazy Sauce*.

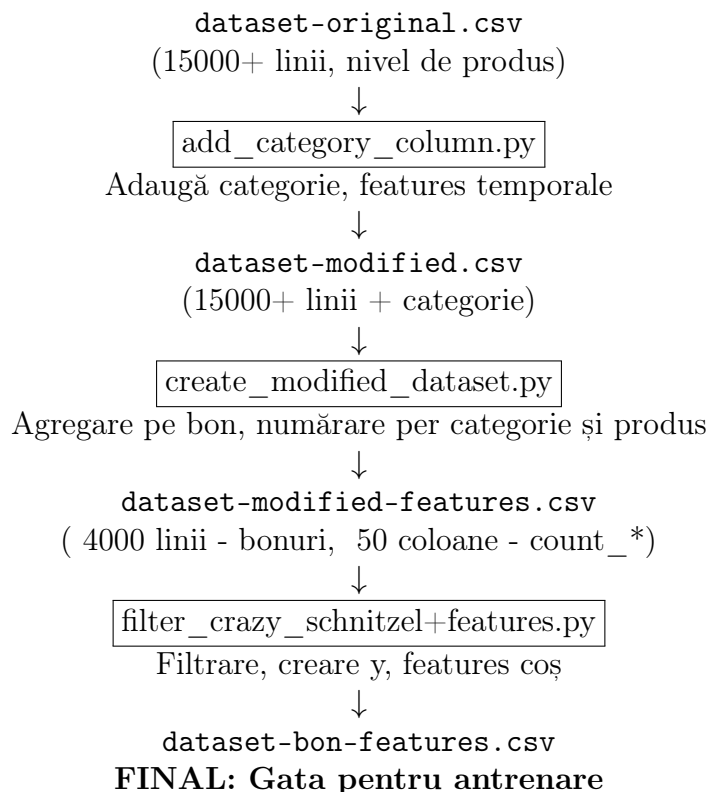
3. Caracteristici de Coș

Pentru fiecare bon, calculez caracteristici agregate:

Tabela 2: Caracteristici de coș

Feature	Descriere
<code>cart_size</code>	Numărul total de produse pe bon
<code>distinct_products</code>	Numărul de produse DIFERITE comandate
<code>total_value</code>	Valoarea totală a comenzii (în bani)
<code>avg_price</code>	Preț mediu per produs
<code>count_category_*</code>	Numărul de produse din fiecare categorie

Rezumat Complet al Fluxului



2 Preprocesare (Task 2.2)

Același flux de preprocesare a fost aplicat și pentru problema 2.2. Diferența majoră a constatat în absența etapei de filtrare, utilizându-se toate bonurile disponibile, nu doar cele care conțineau produsul *Crazy Schnitzel*. Variabila țintă (y) a fost, pe rând, fiecare dintre cele 8 sosuri posibile, la antrenare acesta fiind exclus din trăsăturile furnizate pentru a evita “scurgerile de date” (*data leakage*).

3 Regresie Logistică (Task 2.1)

Această secțiune detaliază procesul iterativ prin care am optimizat implementarea proprie a Regresiei Logistice, deciziile luate privind preprocesarea datelor și selecția hiperparametrilor, precum și comparația finală cu implementarea de referință din `scikit-learn`.

3.1 Implementare finală

Pentru rezolvarea problemei de clasificare binară (predicția sosului *Crazy Sauce*), a fost implementată Regresia Logistică folosind metoda Gradient Descent. Minimizarea erorii s-a realizat folosind funcția de cost Binary Cross-Entropy, aceasta fiind standardul pentru problemele de clasificare binară.

În varianta finală, am implementat mecanismul de regularizare L2 pentru a penaliza ponderile excesive și a evita overfitting-ul. A fost inclus și un mecanism de early stopping setat să se declanșeze după 5 epoci consecutive în care funcția de loss crește, deși acesta nu a fost declanșat pe parcursul experimentelor finale, modelul convergând stabil.

Un aspect crucial pentru eficiența algoritmului de optimizare a fost preprocesarea datelor prin standardizare. Am transformat variabilele de intrare astfel încât să aibă media 0 și deviația

standard 1. Această uniformizare a suprafeței de eroare a permis utilizarea unei rate de învățare agresive ($learning_rate = 0.5$) fără riscul divergenței.

Nu în ultimul rând, stabilitatea numerică la începutul antrenării a fost asigurată prin inițializarea ponderilor folosind metoda Xavier, prevenind astfel saturația prematură a funcției de activare Sigmoid.

3.2 Evoluția experimentului

Performanța modelului a fost obținută printr-un proces iterativ de optimizare, rezultatele fiind monitorizate constant comparativ cu baseline-ul `sklearn`.

În prima fază a experimentelor, am rulat modelul pe datele brute cu regularizare L2 și un $lr = 0.1$. Această abordare a rezultat într-o acuratețe modestă de aproximativ 65.83%, sub-performând masiv față de implementarea de referință (76.47%). Diferența de peste 10% a indicat clar că datele necesită o scalare corespunzătoare.

Un salt major a fost realizat prin adăugarea de trăsături de tip contor (*counter features*). Maparea produselor specifice și generarea unor contoare per categorie au permis modelului să generalizeze comportamentul clienților. Această modificare a propulsat acuratețea la aproximativ 87% , după normalizarea datelor ajungând ulterior la 96.64%.

Ulterior, am setat o rată de învățare agresivă ($learning_rate = 0.25$) combinată cu mecanismul de Early Stopping ($patience = 5$ epoci), însă, în mod surprinzător, acesta nu a fost declanșat pe parcursul experimentelor.

Diferența critică a fost făcută de numărul de iterații. Rularea pentru 10.000 de iterații a adus modelul la paritate cu `sklearn` (99.16%), iar extinderea la 15.000 de iterații a permis depășirea baseline-ului pe metricile punctuale, atingând o acuratețe de 99.44%.

Am mai experimentat și testând limitele absolute ale implementării prin rularea a 50.000 de iterații, am reușit să minimizez funcția de loss până la valoarea de 0.0160, și se putea observa de pe plot-ul funcției de loss că aceasta încă scădea.

Este important de menționat și un experiment care nu a dat rezultatele așteptate: implementarea unui *Learning Rate Decay*. Scăderea treptată a ratei de învățare a degradat performanța la nivelul etapelor intermediare ($Acc \approx 98.32\%$), deoarece rata scădea prea rapid, împiedicând modelul să parcurgă pașii necesari pentru a ajunge în zona de minim global atinsă anterior cu rata constantă.

3.3 Evaluarea Modelului și Interpretarea Rezultatelor

Tabela 3: Performanța Finală: Implementare Proprie vs. Scikit-Learn

Metrică	Custom Implementation	sklearn Baseline	Diferență
Accuracy	0.9944	0.9916	+0.0028%
Precision	0.9896	0.9895	+0.0001%
Recall	1.0000	0.9947	+0.0053%
F1-Score	0.9948	0.9921	+0.0026%
ROC-AUC	0.9967	0.9995	-0.0028%

3.4 Matricea de Confuzie

Pentru a vizualiza performanța modelului pe setul de testare, am generat matricea de confuzie (Figura 1). Aceasta ilustrează capacitatea modelului de a distinge corect între bonurile care conțin *Crazy Sauce* și cele care nu.

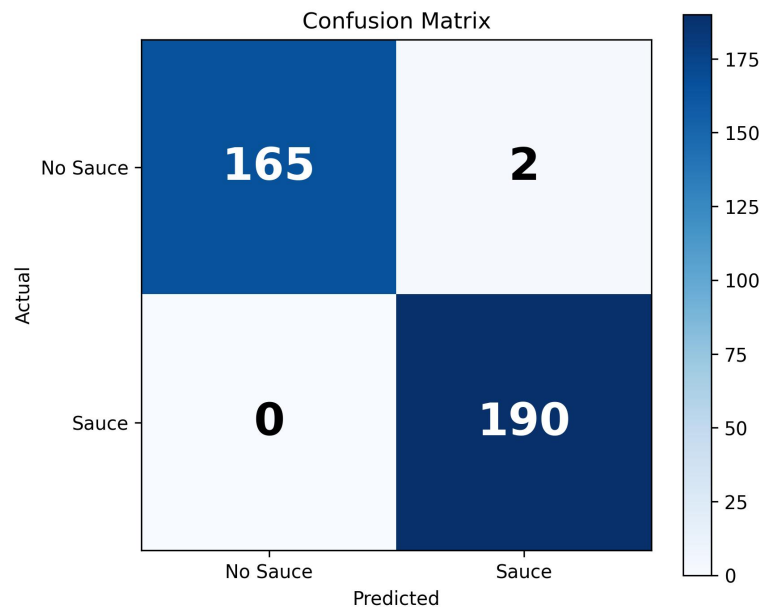


Figura 1: Matricea de Confuzie pentru clasificarea binară a sosului Crazy Sauce.

Rezultatele indică o precizie extrem de ridicată:

- **True Negatives (TN):** 165 (Bonuri fără sos clasificate corect)
- **True Positives (TP):** 190 (Bonuri cu sos clasificate corect)
- **False Positives (FP):** 2 (Bonuri fără sos clasificate greșit ca având sos)
- **False Negatives (FN):** 0 (Niciun bon cu sos nu a fost ratat)

3.5 Analiza Curbei ROC

Graficul demonstrează o capacitate de discriminare aproape ideală pentru ambele modele, curbele tinzând rapid către colțul stânga-sus (Rata de Pozitive Adevărate = 1, Rata de Pozitive False = 0). praguri de discriminare (Figura 2).

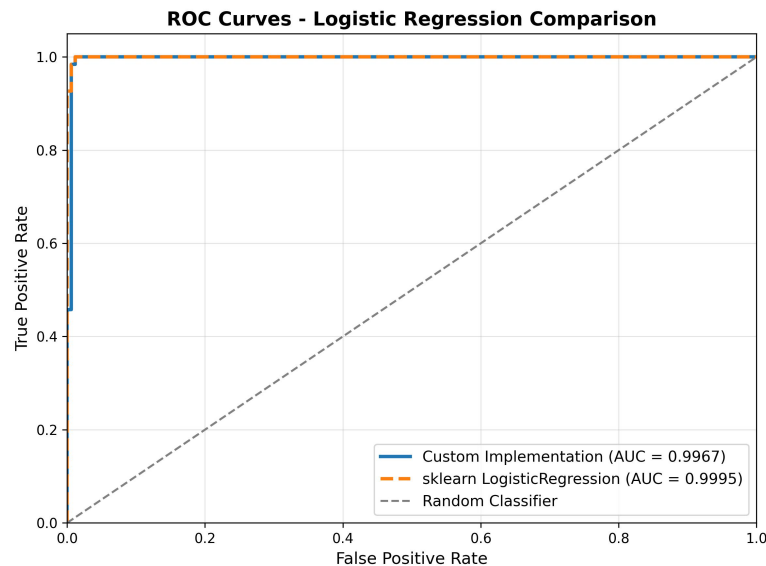


Figura 2: Comparație curbe ROC: Implementare Proprie vs. Scikit-Learn.

3.6 Interpretarea Coeficienților Regresiei Logistice

Un avantaj major al Regresiei Logistice este interpretabilitatea. Analizând ponderile (w) asociate fiecărei trăsături, putem înțelege ce factori influențează decizia clientului de a cumpăra *Crazy Sauce*.

Factori care CRESC probabilitatea de cumpărare (Coeficienți Pozitivi)

- **Mărimea Coșului (`cart_size`, +12.04):** Acesta este de departe cel mai puternic predictor. Cu cât un client comandă mai multe produse, cu atât este mai probabil să comande și un sos.
- **Valoarea Totală (`total_value`, +3.97):** Corelat cu mărimea coșului, clienții care cheltuie mai mult sunt mai dispuși să adauge produse complementare.
- **Diversitatea (`distinct_products`, +1.98):** O varietate mai mare de produse în coș indică o masă complexă, care necesită adesea sosuri.

Factori care SCAD probabilitatea de cumpărare (Coeficienți Negativi)

- **Alte Sosuri (`has_cheddar_sauce`, -4.91; `has_garlic_sauce`, -4.19):** Dacă un client a comandat deja un alt tip de sos, probabilitatea de a comanda și *Crazy Sauce* scade drastic. Aceasta confirmă intuiția că produsele din aceeași categorie sunt substituibile.
- **Prețul Mediu (`avg_price`, -3.10):** Un preț mediu ridicat ar putea indica comanda unor produse scumpe care nu necesită sos (de exemplu, salate complexe sau meniuri care includ deja sosuri).
- **Băuturile (`count_drink`, -1.92):** Clienții care comandă preponderent băuturi (probabil doar o gustare sau o pauză de cafea) sunt mai puțin interesați de sosuri.

4 Sistem de Recomandare Sosuri (Task 2.2)

Extinzând arhitectura validată la punctul anterior, am dezvoltat un sistem de recomandare capabil să sugereze sosul ideal în funcție de componența coșului de cumpărături.

4.1 Abordare și Implementare

Deoarece problema implică recomandarea unuia dintre cele 8 sosuri posibile, am adoptat strategia One-vs-All. Arhitectura de regresie logistică utilizată este identică cu cea de la 2.1, însă aplicată pe setul extins de date.

Diferențe față de Task 2.1

Spre deosebire de abordarea anterioară, implementarea pentru sistemul de recomandare a necesitat câteva ajustări, majoritatea la nivel de preprocesare. A fost folosit datasetul extins (nu doar bonurile ce conțin Crazy Schnitzel).

Arhitectura a fost adaptată prin antrenarea a 8 modele distincte, câte unul pentru fiecare tip de sos (*Crazy Sauce*, *Cheddar*, *Garlic*, etc.), trecând astfel la o strategie de tip One-vs-All. Înainte de antrenarea fiecărui sos a fost eliminată coloana caracteristică sosului curent pentru a nu avea scurgeri de date.

4.2 Rezultate Experimentale

Performanța la Nivel de Clasificator Individual

Fiecare dintre cele 8 modele a converș după 15000 de iterații ($\max(\text{Loss}) \approx 0.0024$).

Tabela 4: Acuratețea modelelor individuale per tip de sos

Tip Sos	Final Loss	Accuracy
Crazy Sauce	0.0022	0.9994
Cheddar Sauce	0.0024	1.0000
Extra Cheddar Sauce	0.0007	1.0000
Garlic Sauce	0.0023	0.9987
Tomato Sauce	0.0024	0.9994
Blueberry Sauce	0.0024	1.0000
Spicy Sauce	0.0024	1.0000
Pink Sauce	0.0023	0.9987

Performanța Sistemului de Recomandare

Pentru evaluare, am comparat sistemul nostru cu un **Baseline de Popularitate**. Metrica principală, **Hit@K**, măsoară dacă sosul real cumpărat de client se află în primele K sugestii ale modelului.

4.3 Interpretarea Rezultatelor

Rezultatele obținute sunt categorice:

Tabela 5: Comparație Model Recomandare vs. Baseline Popularitate globală

Metrică	K	Model Propriu	Baseline (Popularitate)	Improvement
Hit@1	1	1.0000	0.7975	+20.25%
Hit@3	3	1.0000	0.7975	+20.25%
Hit@5	5	1.0000	1.0000	+0.00%
Precision@1	1	1.0000	0.7975	+20.25%
Precision@3	3	0.3333	0.2658	+6.75%
Precision@5	5	0.2000	0.2000	+0.00%

- **Hit@1 (1.0000):** Modelul a reușit să prezică corect sosul ales de client *de fiecare dată* pe prima poziție. Aceasta indică faptul că, în datele analizate, alegerea sosului este puternic determinată în funcție de produsele din coș (ex: *Blueberry Sauce* este probabil asociat exclusiv cu un desert specific sau un produs de tip "Papanasi").
- **Superioritate față de Baseline:** Deși popularitatea generală oferă o ghicire bună (79.75%), modelul acoperă "golul" de 20%, personalizând perfect recomandarea pentru cazurile mai rare sau specifice.